

Unit 2

Divide and conquer & Greedy Strategy

Syllabus

- Divide and conquer - Merge Sort, Quick sort, Large Integer multiplication, solving recurrences (substitution method & Master's theorem).
- Greedy strategy: Principle, control abstraction, Knapsack problem, Job sequencing with Deadlines, Huffman Encoding

Optimization problems

- An **optimization problem** is one in which you want to find, not just *a* solution, but the *best* solution
- A “greedy algorithm” sometimes works well for optimization problems
- A **greedy algorithm** works in phases. At each phase:
 - You take the best you can get right now, without regard for future consequences
 - You hope that by choosing a *local* optimum at each step, you will end up at a *global* optimum

Introduction : Greedy Method

- A **greedy algorithm** for an optimization problem always makes the choice that **looks best at the moment** and adds it to the current subsolution.
- Final output is an optimal solution.
- Greedy algorithms don't always yield optimal solutions but, when they do, they're usually the simplest and most efficient algorithms available.

Introduction : Greedy Method

- It is used to solve problems that have 'n' inputs and require us to obtain a subset that **satisfies some constraints**.
- Any subset that satisfied the constraints is called as a **feasible** solution.
- We need to find the optimum feasible solution i.e. the feasible solution that optimizes the given **objective functions**.
- It works in stages. At each stage, At each stage a decision is made regarding whether or particular input is in the optimum solution.

Introduction : Greedy Method

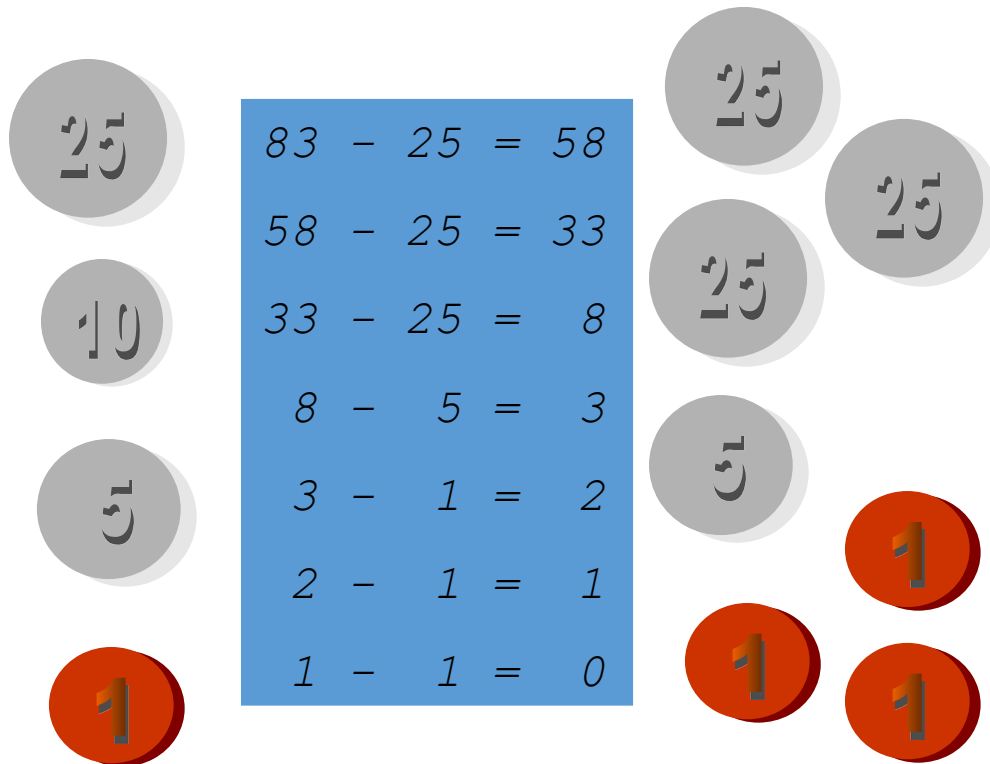
- The greedy method suggests that one can divide the algorithm that works in stages. Considering one input at a time.
- At each stage a decision is made regarding whether or particular input is in the optimum solution.
- For this purpose all the inputs must be arranged **in a particular order by using a selection process**.
- If the inclusion of next input in to the partially constructed solution will result in an infeasible solution then the input will not be considered and will not be added to partially constructed set otherwise it is added.
- **CHANGE-MAKING PROBLEM (coin changing)**

Coin Changing

An optimal solution to the coin changing problem is the minimum number of coins whose total value equals a specified amount. For example what is the minimum number of coins (current U.S. mint) needed to total 83 cents.

Objective function: Minimize number of coins returned.

Greedy solution: Always return the largest coin you can



A Greedy Algorithm for Coin Changing

1. Set `remval=initial_value`
2. Choose largest coin that is less than `remval`.
3. Add coin to set of coins and set `remval:=remval-coin_value`
4. repeat Steps 2 and 3 until `remval = 0`:

Greedy Method

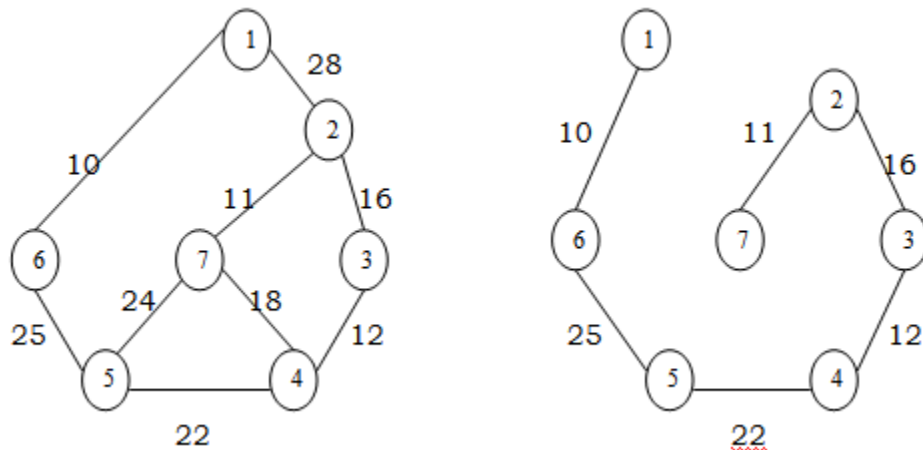
- **Feasible Solution:-**

Any subset of the solutions that satisfies the constraints of the problem is known as a feasible solution.

- **Optimal Solution:-**

The feasible solution that maximizes or minimizes the given objective function is an optimal solution.

Example : MST



Control Abstraction : Greedy Method

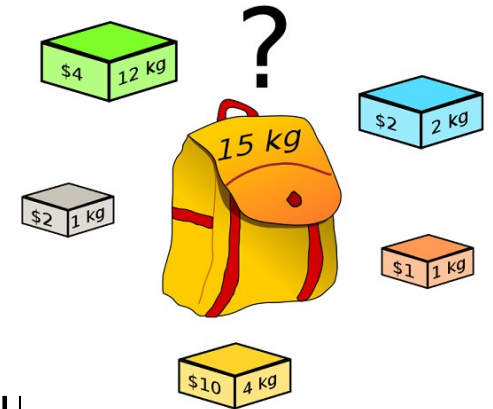
```
Algorithm Greedy (a,n)
//a[1:n] contains the 'n' inputs
{
    Solution: = 0; //initialize the solution
    for i: = 1 to n do
    {
        x: = select (a);
        if Feasible (solution, x) then
            solution: = Union (solution, x);
    }
    return solution;
}
```

- **Select** is a function which is used to select an input from the set.
- **Feasible** is a function which verifies the constraints and determines whether the resultant solution is feasible or not.
- **Union** is a function which is used to add elements to the partially constructed set.

Knapsack Problem

- **Given :**

- A knapsack with capacity ' m '
- A set of ' n ' objects with each item i having
- p_i - a positive benefit
- w_i - a positive weight



- **Goal:** Choose items with maximum total benefit but with weight at most m .

- If we are allowed to take fractional amounts, then this is the **fractional knapsack problem**.

- Let x_i denote the amount we take of item i , $0 \leq x_i \leq 1$

- Objective: **maximize** $\sum_{1 \leq i \leq n} p_i x_i$

- Constraint: $\sum_{1 \leq i \leq n} w_i x_i \leq m$

and $0 \leq x_i \leq 1$, $1 \leq i \leq n$

Knapsack Problem : Example

Ex: - Consider following instance of Knapsack Problem with 3 objects whose profits and weights are defined as

$$(P_1, P_2, P_3) = (25, 24, 15) \quad (W_1, W_2, W_3) = (18, 15, 10)$$

$n=3$ Knapsack Capacity $m=20$

Determine the optimum strategy for placing the objects in to the knapsack.

The four feasible solutions are :

(1) Greedy about profit: -

(x_1, x_2, x_3)	Profit Order	Weight Order	$\sum x_i w_i \leq 20$	$\sum x_i p_i$
$(1, 2/15, 0)$	(p_1, p_2, p_3) $(25, 24, 15)$	(w_1, w_2, w_3) $(18, 15, 10)$	$18 \times 1 + (2/15) \times 15 = 20$	$25 \times 1 + (2/15) \times 24 = 28.2$

(2) Greedy about weight: -

(x_1, x_2, x_3)	Profit Order	Weight Order	$\sum x_i w_i \leq 20$	$\sum x_i p_i$
$(0, 2/3, 1)$	(p_3, p_2, p_1) $(15, 24, 25)$	(w_3, w_2, w_1) $(10, 15, 18)$	$18 \times 0 + (2/3) \times 15 + 10 = 20$	$25 \times 0 + (2/3) \times 24 + 25 = 31$

(3) Greedy about profit / unit weight: -

(x_1, x_2, x_3)	$\sum x_i w_i \leq 20$	$\sum x_i p_i$
$(0, 1, \frac{1}{2})$	$18 \times 0 + 1 \times 15 + (\frac{1}{2}) \times 10 = 20$	$25 \times 0 + 1 \times 24 + (\frac{1}{2}) \times 15 = 31.5$

(4) If an additional constraint of including each and every object is placed then the greedy strategy could be

$$(\frac{1}{2}, \frac{1}{3}, \frac{1}{4}) \quad \sum x_i w_i = \frac{1}{2} \times 18 + \frac{1}{3} \times 15 + \frac{1}{4} \times 10 = 16.5$$

$$\sum x_i p_i = \frac{1}{2} \times 25 + \frac{1}{3} \times 24 + \frac{1}{4} \times 15 = 12.5 + 8 + 3.75 = 24.25$$

(x_1, x_2, x_3)	$\sum x_i w_i \leq 20$	$\sum x_i p_i$
$(\frac{1}{2}, \frac{1}{3}, \frac{1}{4})$	16.5	24.25

Algorithm : Greedy Knapsack

Algorithm Greedy knapsack (m, n)

//p (1:n) and w(1:n) contain the profits and weights resp. of the n objects

//ordered such that $p(i)/w(i) \geq p(i+1)/w(i+1)$. M is the knapsack size and x (1:n)

// is the solution vector

```
{
  for i: = 1 to n do x(i) = 0.0 // initialize x           O(n)
  u := m;
  for i: = 1 to n do                                     O(n)
  {
    if (w(i) > u) then break;
    x(i): = 1.0;
    u: = u-w(i);
  }
  if (i ≤ n) then x(i): = u/w(i);
}
```

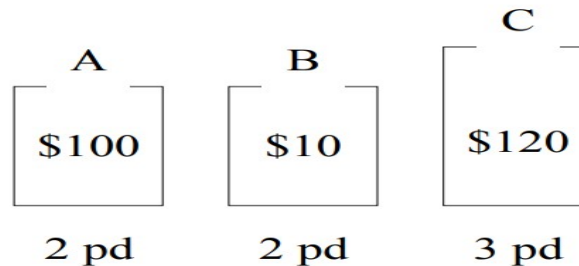
- **Analysis:** - If. we do not consider the time considered for sorting the inputs then the complexity will be **$O(n)$** .

Knapsack : More examples

1) Greedy approach- consider the following instances of knapsack problem
 $n=5$, $w=100$, W (10, 20, 30, 40, 50) , V (20, 30, 66, 40, 60), find the optimal solution.

2)

A thief enters a store and sees the following items:



His Knapsack holds 4 pounds. What should he steal to maximize profit?

3) Let us consider that the capacity of the knapsack $W=60$ and the list of provided items are shown in the following table -

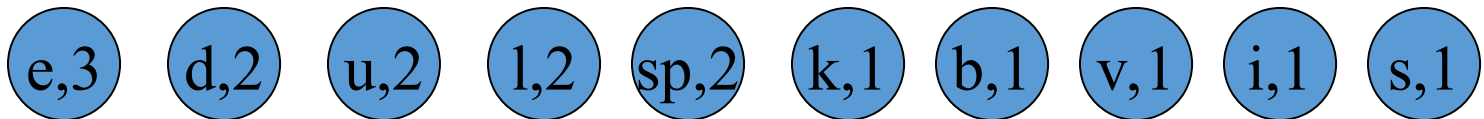
Item	A	B	C	D
Profit	280	100	120	120
Weight	40	10	20	24

Huffman Coding

- Huffman codes can be used to compress information
 - JPEGs do use Huffman as part of their compression process
- The basic idea is to store the more frequently occurring characters using fewer bits and less frequently occurring characters using more bits
 - On average this should decrease the filesize (usually $\frac{1}{2}$)

Huffman Coding

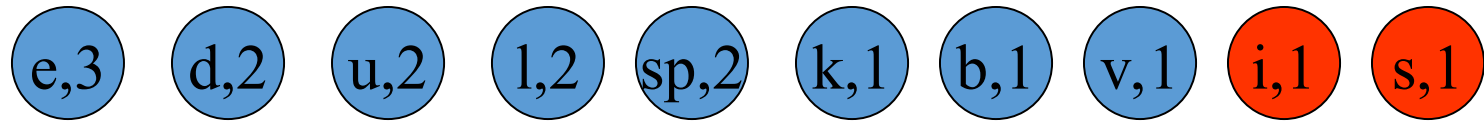
- As an example, lets take the string:
“duke blue devils”
- We first do a frequency count of the characters:
 - e:3, d:2, u:2, l:2, space:2, k:1, b:1, v:1, i:1, s:1
- Next we use a Greedy algorithm to build up a Huffman Tree
 - We start with nodes for each character



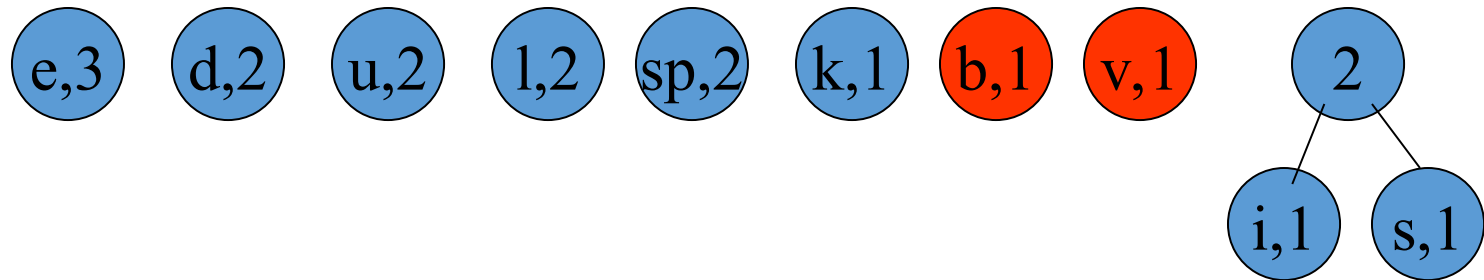
Huffman Coding

- We then pick the nodes with the smallest frequency and combine them together to form a new node
 - The selection of these nodes is the Greedy part
- The two selected nodes are removed from the set, but replace by the combined node
- This continues until we have only 1 node left in the set

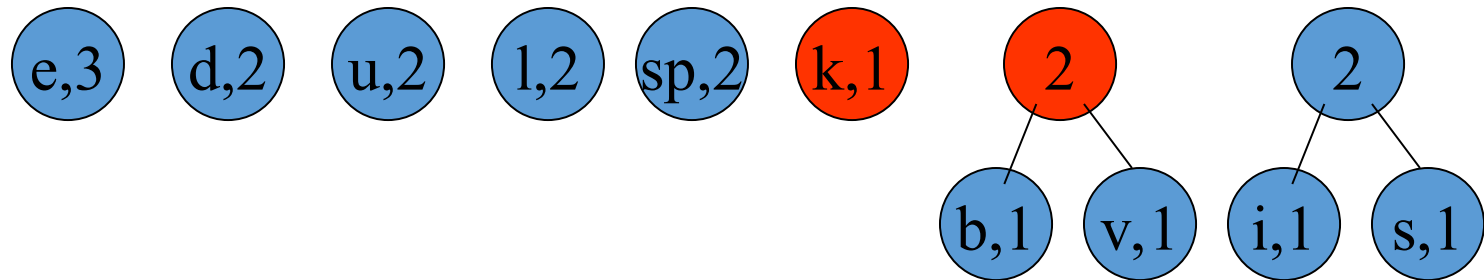
Huffman Coding



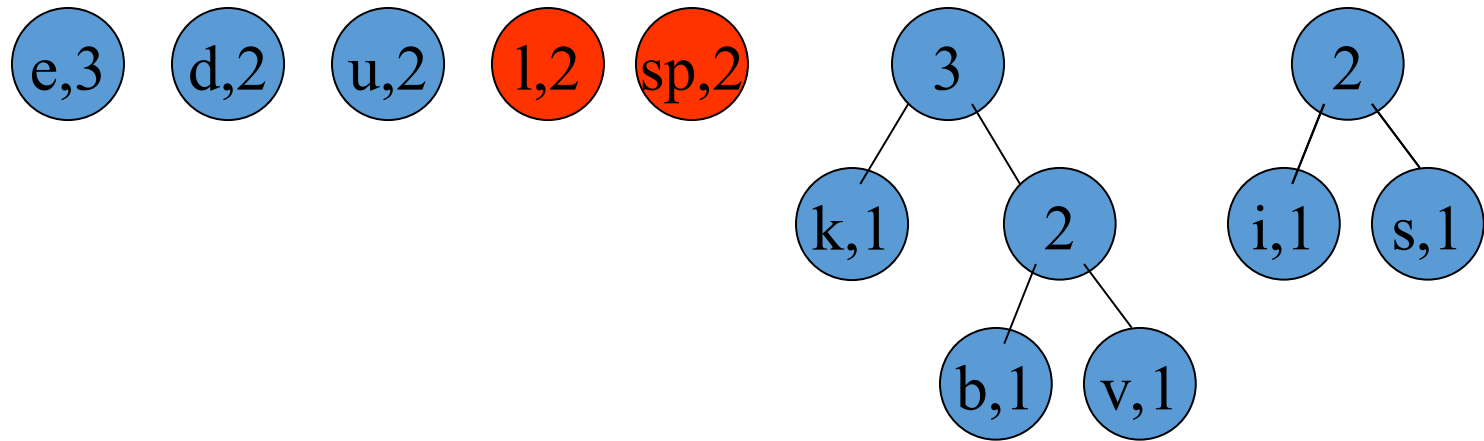
Huffman Coding



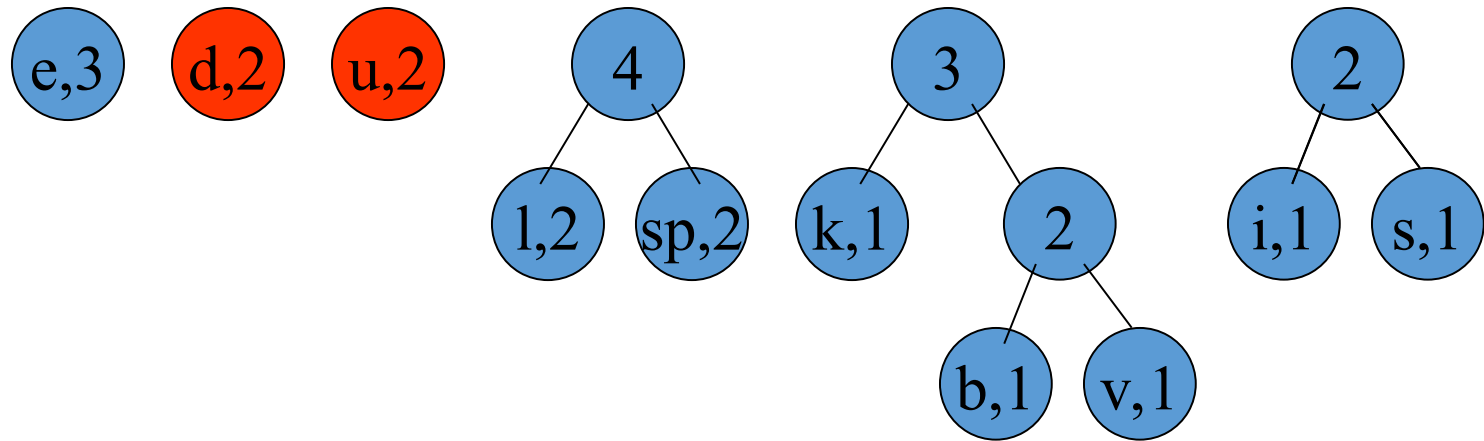
Huffman Coding



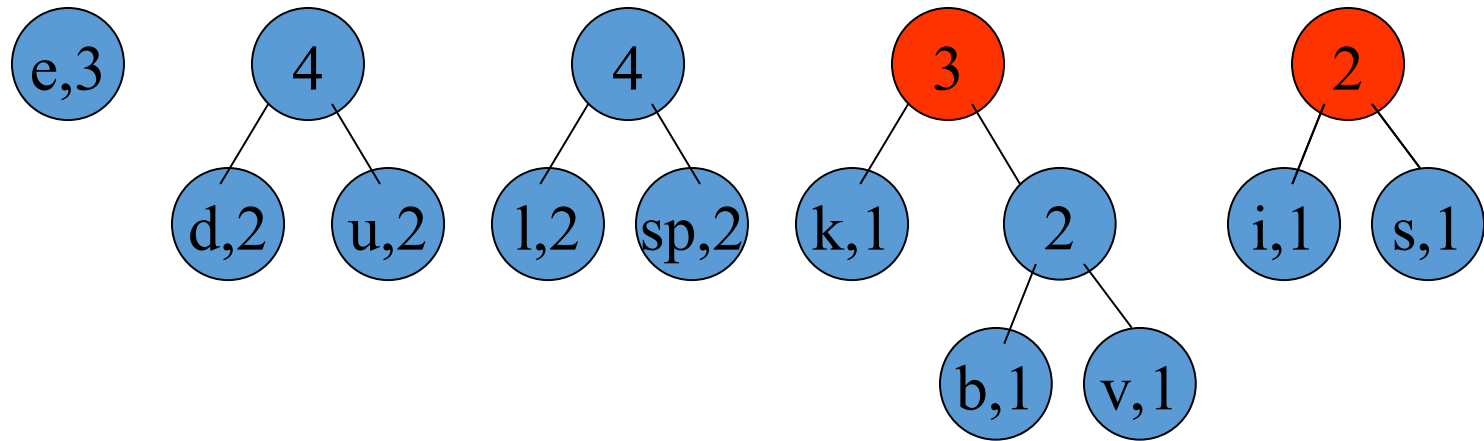
Huffman Coding



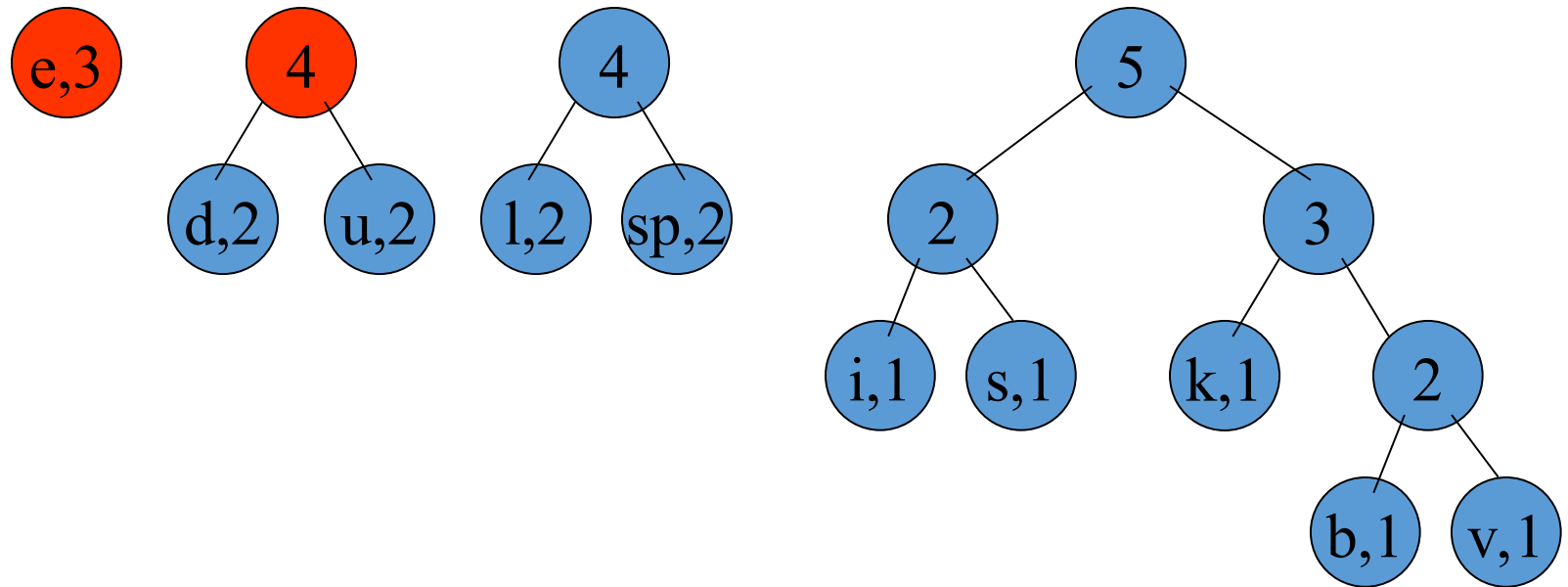
Huffman Coding



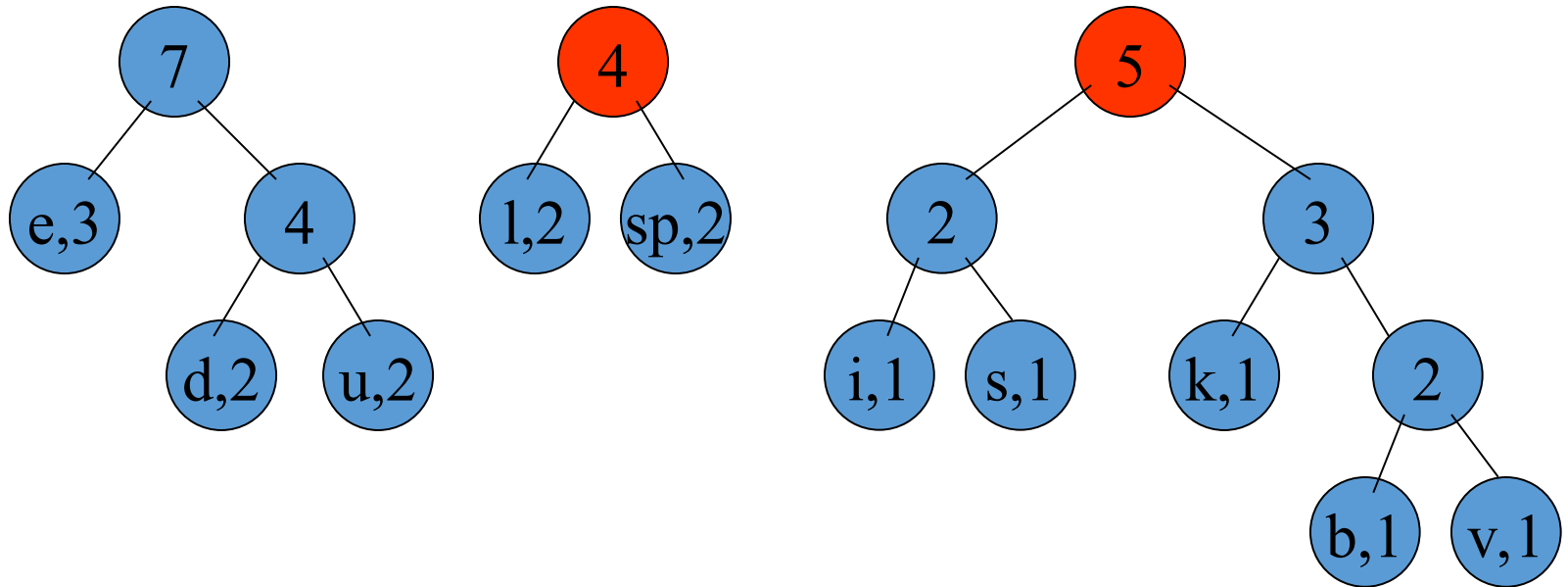
Huffman Coding



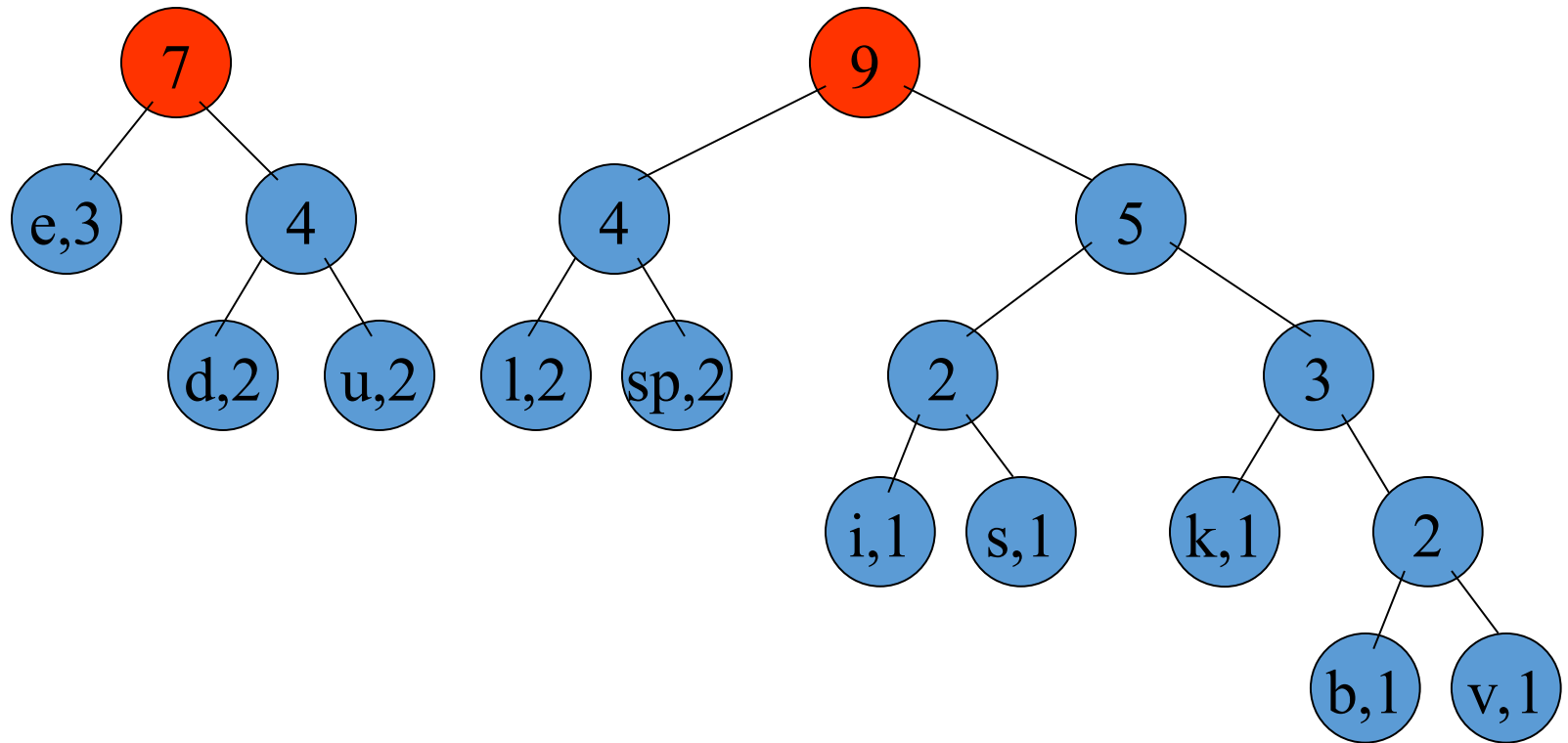
Huffman Coding



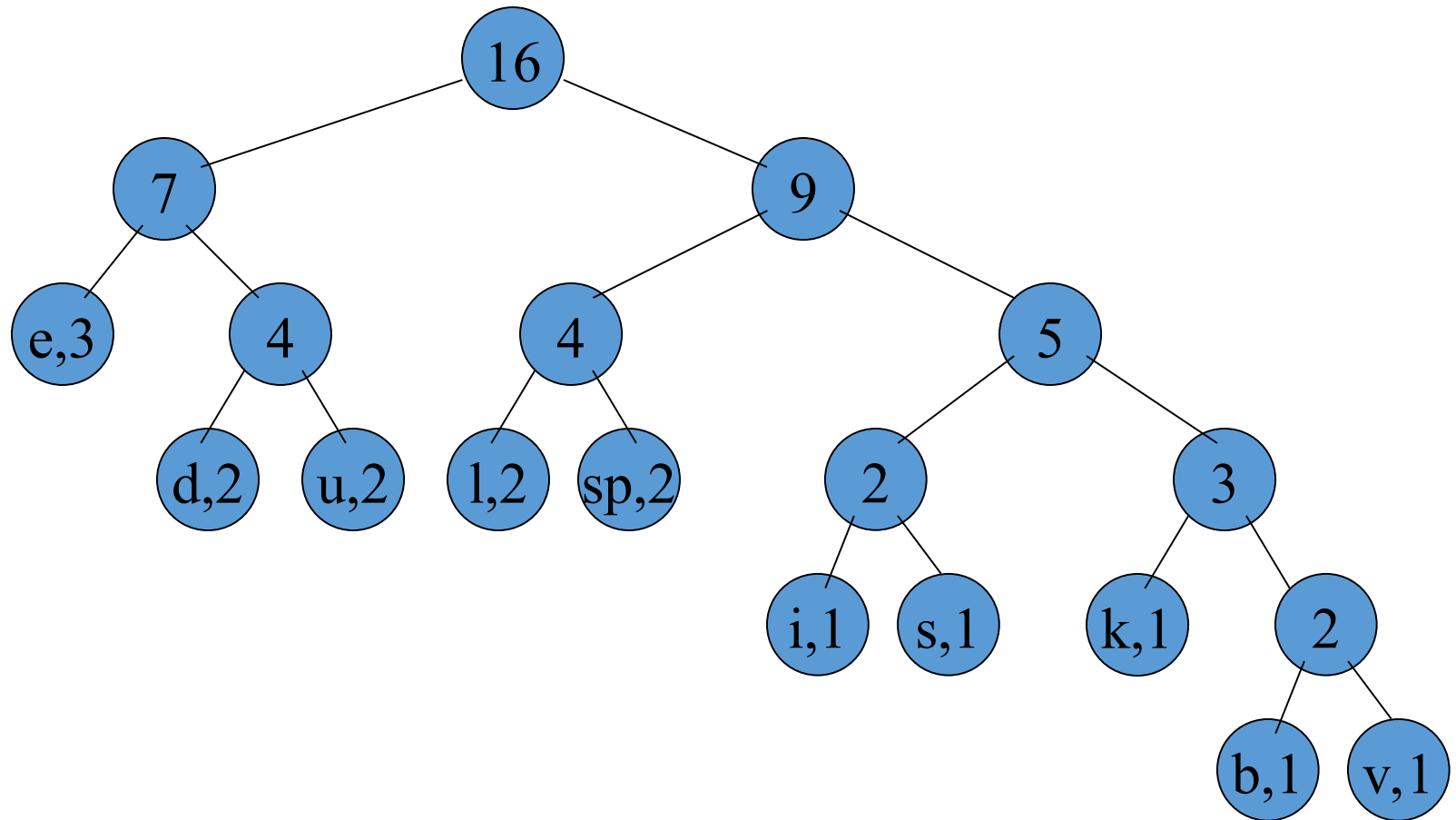
Huffman Coding



Huffman Coding



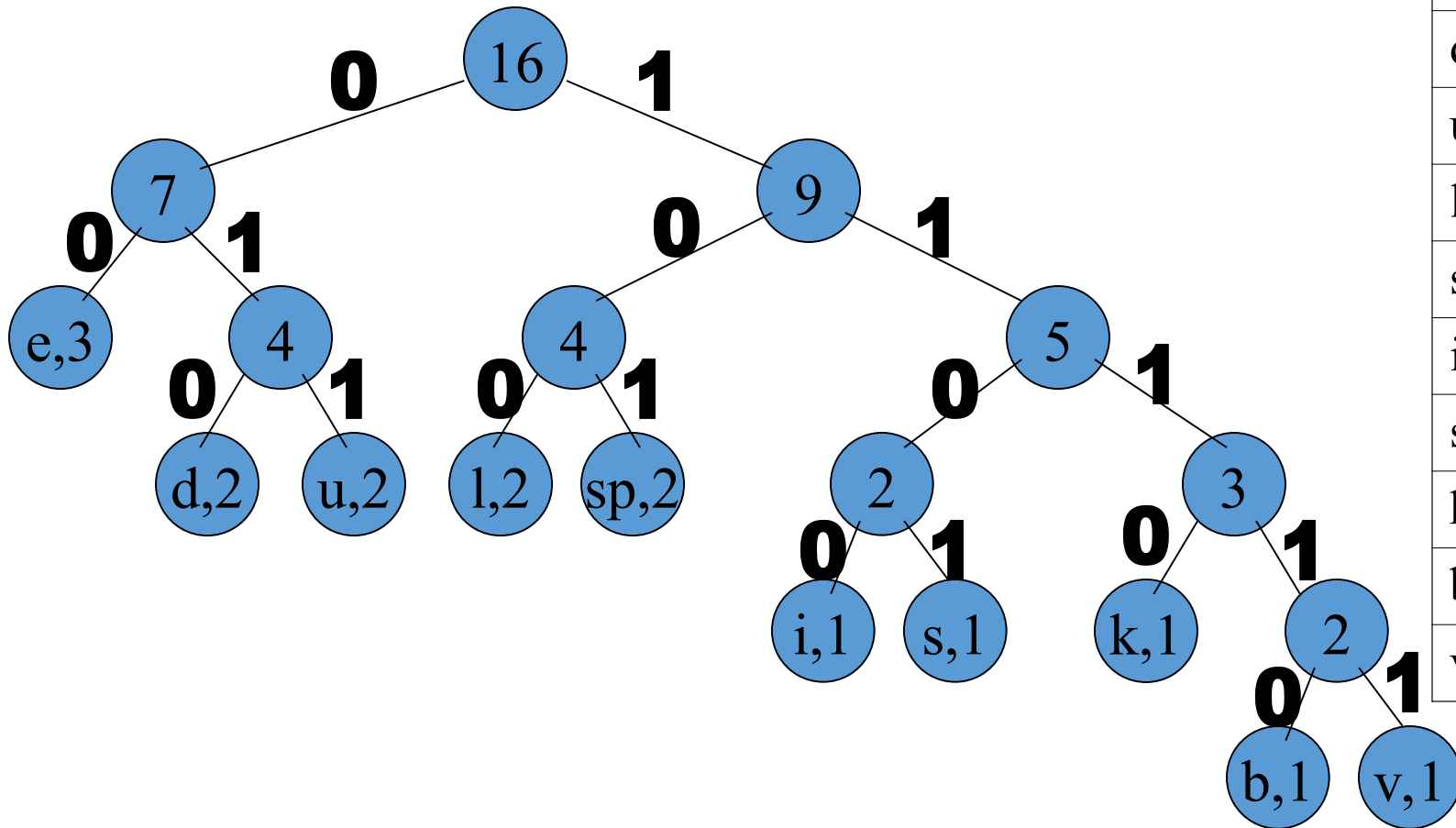
Huffman Coding



Huffman Coding

- Now we assign codes to the tree by placing a 0 on every left branch and a 1 on every right branch
- A traversal of the tree from root to leaf give the Huffman code for that particular leaf character
- Note that no code is the prefix of another code

Huffman Coding



e	00
d	010
u	011
l	100
sp	101
i	1100
s	1101
k	1110
b	11110
v	11111

* Using variable length encoding

Huffman Encoding cormen

HUFFMAN (C)

1. $n = |C|$

//initializes the min-priority queue Q with the characters in C

2. $Q = C$ $O(n)$

//loop extracts the two nodes x and y of lowest frequency from the queue

3. **for i = 1 to n - 1** $O(n \log n)$

//replacing them in the queue with a new node z representing their merger

4. allocate a new node z

5. $z.left = x = \text{EXTRACT-MIN}(Q)$ $O(\log n)$

6. $z.right = y = \text{EXTRACT-MIN}(Q)$

7. $z.freq = x.freq + y.freq$

8. INSERT (Q,z)

//After n-1 mergers, line 9 returns the one node left in the queue, which is the root of the code tree

9. **return EXTRACT-MIN(Q) // return the root of the tree**

Analysis : Must sort n values before making n choices. Therefore, Huffman is $O(n \log n) + O(n) = O(n \log n)$

Huffman Coding

- These codes are then used to encode the string

- Thus, “duke blue devils” turns into:

010 011 1110 00 101 11110 100 011 00 101 010 00 11111 1100 100 1101

- When grouped into 8-bit bytes:

01001111 10001011 11101000 11001010 10001111 11100100 1101xxxx

- Thus it takes 7 bytes of space compared to 16 characters * 1 byte/char = 16 bytes uncompressed

Examples : Huffman Coding

- Solve the following using Huffman's code generation algorithm

Symbol	Probability
a	0.12
b	0.04
c	0.45
d	0.16
e	0.23

Huffman Coding Algorithm Example

Construct a Huffman tree by using these nodes

Value	A	B	C	D	E	F
Freq uenc y	5	25	7	15	4	12

Other greedy algorithms

- Dijkstra's algorithm for finding the shortest path in a graph
 - Always takes the *shortest* edge connecting a known node to an unknown node
- Kruskal's algorithm for finding a minimum-cost spanning tree
 - Always tries the *lowest-cost* remaining edge
- Prim's algorithm for finding a minimum-cost spanning tree
 - Always takes the *lowest-cost* edge between nodes in the spanning tree and nodes not yet in the spanning tree

JOB SEQUENCING WITH DEADLINES

The problem is stated as below.

- There are n jobs to be processed on a machine.
- Each job i has a deadline $d_i \geq 0$ and profit $p_i \geq 0$.
- p_i is earned iff the job is completed by its deadline.
- The job is completed if it is processed on a machine for unit time.
- Only one machine is available for processing jobs.
- Only one job is processed at a time on the machine.

JOB SEQUENCING WITH DEADLINES (contd..)

- A feasible solution is a subset of jobs J such that each job is completed by its deadline.
- An optimal solution is a feasible solution with maximum profit value.
- NO LATER THAN THEIR RESPECTIVE DEADLINES.

Example : Let $n = 4$, $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$,
 $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$

JOB SEQUENCING WITH DEADLINES (contd..)

Example : Let $n = 4$, $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$,
 $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$

Sr.No.	Feasible Solution	Processing Sequence	Profit value
(i)	(1,2)	(2,1)	110
(ii)	(1,3)	(1,3) or (3,1)	115
(iii)	(1,4)	(4,1)	127
(iv)	(2,3)	(2,3)	25
(v)	(3,4)	(4,3)	42
(vi)	(1)	(1)	100
(vii)	(2)	(2)	10
(viii)	(3)	(3)	15
(ix)	(4)	(4)	27

127 is the optimal one

Example Explanation

- Consider the jobs in the non increasing order of profits subject to the constraint that the resulting job sequence J is a feasible solution.
- In the example considered before, the non-increasing profit vector is

(100	27	15	10)	(2	1	2	1)	p_1	p_4	p_3
	p_2		d_1	d_4		d_3	d_2			

GREEDY ALGORITHM TO OBTAIN AN OPTIMAL SOLUTION (Contd..)

(100	27	15	10)	(2	1	2	1)
p_1	p_4	p_3	p_2	d_1	d_4	d_3	d_2

$J = \{ 1 \}$ is a feasible one

$J = \{ 1, 4 \}$ is a feasible one with processing
sequence (4,1)

$J = \{ 1, 3, 4 \}$ is not feasible

$J = \{ 1, 2, 4 \}$ is not feasible

$J = \{ 1, 4 \}$ is optimal

Examples :Job Sequencing Problems:

1) Write a greedy algorithm for sequencing unit time jobs with deadlines and profits. Using this algorithm, find the optimal solutions when $n=5, (p_1, p_2, p_3, p_4, p_5)=(20, 15, 10, 5, 1)$ and $(d_1, d_2, d_3, d_4, d_5)=(2, 2, 1, 3, 3)$.

2) Find the correct sequence for jobs using following instances,

JobID	Deadline	Profit
1	4	20
2	1	10
3	1	40
4	1	30

3) Find the correct sequence for jobs using following instances,

JobID	Deadline	Profit
1	2	100
2	1	19
3	2	27
4	1	25
•	3	15

4) Find the optimum job sequence for the following problem.

$n = 7$ P_7 $J_1 \ J_2 \ J_3 \ J_4 \ J_5 \ J_6 \ J_7$
 $(P_1, P_2, P_3, P_4, P_5, P_6)$ = $(3, 5, 10, 18, 1, 6, 30)$
 $, d_4, d_5, d_6)$ = $(1, 3, 4, 3, 2, 1, 2)$

Feasible soln.

$(1, 2, 3, 4)$

Processing Sequence

$(1, 2, 4, 3)$ or $(1, 4, 2, 3)$

Value

$3+5+10+18= 36$

Job sequencing with deadline

Example -

index	1	2	3	4	5
Jobs	A ₁	A ₂	A ₃	A ₄	A ₅
deadlines	2	1	3	2	1
Profit	60	100	20	40	20

Job sequencing with deadline

⇒ sort the jobs according to profit

index	1	2	3	4	5
jobs	A₂	A ₁	A ₄	A ₃	A ₅
deadlines	1	2	2	3	1
Profit	100	60	40	20	20

Job sequencing with deadline

Pseudo code:-

for $i = 1$ to n do

set $k = \min(d_{\max}, \text{Deadline}(i))$

while $k \geq 1$ do

if time slot $[k]$ is empty then

time slot $[k] = \text{job}[i]$

break.

End if

set $k = k - 1$

End while

End for.

GREEDY ALGORITHM TO OBTAIN AN OPTIMAL SOLUTION (Contd..)

$J(r+1) \Leftarrow i ; k \Leftarrow k+1$

// i is inserted at position r+1 //

// and total jobs in J are increased by one //

repeat

end JS

Algorithm for Job sequencing with Deadlines

Algorithm JS(d, j, n)

// $d[i] \geq 1$, $1 \leq i \leq n$ are the deadlines, $n \geq 1$. The jobs
// are ordered such that $p[1] \geq p[2] \geq \dots \geq p[n]$. $J[i]$
// is the i th job in the optimal solution, $1 \leq i \leq k$.
// Also, at termination $d[J[i]] \leq d[J[i+1]]$, $1 \leq i < k$.

{

$d[0] := J[0] := 0$; // Initialize.

$J[1] := 1$; // Include job 1.

$k := 1$;

for $i := 2$ **to** n **do**

 {

 // Consider jobs in nonincreasing order of $p[i]$. Find
 // position for i and check feasibility of insertion.

$r := k$;

while $((d[J[r]] > d[i])$ **and** $(d[J[r]] \neq r))$ **do** $r := r - 1$;

if $((d[J[r]] \leq d[i])$ **and** $(d[i] > r))$ **then**

 {

 // Insert i into $J[]$.

for $q := k$ **to** $(r + 1)$ **step** -1 **do** $J[q + 1] := J[q]$;

$J[r + 1] := i$; $k := k + 1$;

 }

 }

return k ;

}

COMPLEXITY ANALYSIS OF JS ALGORITHM

- Let n be the number of jobs and s be the number of jobs included in the solution.
- The loop between lines 4-15 (the for-loop) is iterated $(n-1)$ times.
- Each iteration takes $O(k)$ where k is the number of existing jobs.
- ∴ The time needed by the algorithm is $O(sn)$ $s \leq n$ so the worst case time is $O(n^2)$.

If $d_i = n - i + 1$ $1 \leq i \leq n$, JS takes $\theta(n^2)$ time

D and J need $\theta(s)$ amount of space.